Gym Tracker Pro

Task 2 - Testing I - Summary

Introduction

This document presents a testing summary for the GymTracker Pro backend application (/lib/ directory), covering Black-Box Testing (BBT), White-Box Testing (WBT) and Integration Testing (IT).

All tests were implemented and executed using the Jest testing framework with Istanbul code coverage reporting. The final test run achieved 100% coverage across statements, branches, functions, and lines, confirming complete execution coverage of the backend codebase.

The following sections detail the applied testing methodologies, observed findings, and final validation results.

All files

100% Statements `632/632` **100%** Branches `261/261` **100%** Functions `119/119` **100%** Lines `596/596`

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
lib	100%	6/6	100%	0/0	100%	2/2	100%	6/6
lib/controller	100%	258/258	100%	90/90	100%	29/29	100%	257/257
lib/domain	100%	13/13	100%	0/0	100%	1/1	100%	10/10
lib/repository	100%	181/181	100%	153/153	100%	39/39	100%	176/176
lib/schema	100%	55/55	100%	0/0	100%	1/1	100%	35/35
lib/service	100%	113/113	100%	18/18	100%	47/47	100%	109/109
prisma/generated/prisma	100%	6/6	100%	0/0	100%	0/0	100%	3/3

Code coverage generated by istanbul at 2026-04-25T15:49:14.369Z

Black-Box Testing

Overview

This section describes the Black-Box Testing (BBT) effort carried out on the backend layer of the GymTracker Pro application, located under gymtrackerpro/lib/.

The testing was performed without reliance on internal implementation details, operating exclusively from the observable specification of each component's specification, interface, inputs, pre-conditions, expected outputs, and post-conditions.

Two domain testing techniques were applied systematically throughout:

- **Equivalence Class (EC)** partitioning: the input domain of each function is partitioned into equivalence classes (valid and invalid) such that a test of one representative element of a class is deemed equivalent to testing any other element of the same class.
- **Boundary Value Analysis (BVA)**: test cases are derived from the boundaries of each equivalence class: values directly on, just below, and just above each edge.

For ranged numeric or date inputs, one valid EC and two invalid ECs (below minimum and above maximum) were identified, yielding boundary test points at the endpoints and their immediate neighbours. For set-valued inputs, one valid EC per accepted value and one combined invalid EC were defined. For "must-be" (boolean / presence) conditions, one valid and one invalid EC were identified. Where there was any reason to believe that elements within a class were not handled identically by the implementation, the class was further split.

Scope

Testing covered the entire backend stack across four architectural layers:

Layer	Modules under test
Schema	user-schema, exercise-schema, workout-session-schema, report-schema, utils
Repository	user-repository, exercise-repository, workout-session-repository
Service	user-service, exercise-service, workout-session-service, report-service, auth-service
Controller	user-controller, exercise-controller, workout-session-controller, report-controller, auth-controller
Utilities	escapeLike

Each layer was tested independently, with dependencies at lower layers mocked where applicable (e.g., Prisma client mocked in repository tests; repositories mocked in service tests; services mocked in controller tests), ensuring that each suite exercises only the behaviour of its own layer against its published interface.

Test Artifacts

Test Files

A total of 19 test files (.test.ts) were authored, one per module under test.

Test file	Test Count
user-controller.test.ts	156
user-service.test.ts	124
user-repository.test.ts	116
user-schema.test.ts	257
auth-controller.test.ts	20
auth-service.test.ts	11

exercise-controller.test.ts	102
exercise-service.test.ts	77
exercise-repository.test.ts	72
exercise-schema.test.ts	58
workout-session-controller.test.ts	83
workout-session-service.test.ts	47
workout-session-repository.test.ts	59
workout-session-schema.test.ts	152
report-controller.test.ts	21
report-service.test.ts	12
report-schema.test.ts	12
schema/utils.test.ts	55
lib/utils.test.ts	18
TOTAL	**1452**

XLSX Documentation Files

Each function under test was accompanied by a structured BBT form in XLSX format, recording equivalence classes, boundary values, test case identifiers, inputs, expected results, and pass/fail status. A total of 103 XLSX forms were produced.

Layer	Test Count
Schema	20
Repository	24
Service	29
Controller	29
Utilities	1
TOTAL	**103**

Methodology

Equivalence Class Partitioning

For each input parameter of a function, equivalence classes were identified following the rules from:

- Range inputs (e.g., membershipStart date, string length constraints) → 1 valid EC, 2 invalid ECs (below-minimum, above-maximum).
- Set-valued inputs (e.g., MuscleGroup enum, ExerciseCategory enum) → 1 valid EC per accepted value, 1 invalid EC for values outside the set.
- Must-be conditions (e.g., email must match RFC-5322 format, phone must match E.164) → 1 valid EC, 1 invalid EC.
- Null / empty / blank inputs were always treated as a distinct invalid EC.

Valid ECs were combined into a single test case where possible; each invalid EC was exercised by its own dedicated test case.

Boundary Value Analysis

BVA was applied on top of EC partitioning for all inputs with an ordered domain. Boundary test points were:

- The minimum and maximum of the valid range.
- One value below, one value at, and one value above the minimum.
- One value below, one value at, and one value above the maximum.

For string-length constraints this was realised as strings of exactly min-1, min, min+1, max-1, max, and max+1 characters. For set-based every possible value was considered.

Observations and Findings

The comprehensive scope of the testing effort (1452 test cases across 103 forms) surfaced several gaps that would not have been caught by inspection alone:

- Incomplete or ambiguous specifications. Multiple input fields lacked explicit minimum or maximum length constraints in the original problem statement. Applying EC partitioning forced the identification of class boundaries, revealing that the specification needed to be updated to make these limits explicit.
- Undocumented enum values. BBT for set-valued inputs (e.g., MuscleGroup, ExerciseCategory) made it apparent that not all accepted values were listed in the original documentation.
- Inconsistent handling of boundary dates. BVA on date fields (e.g., dateOfBirth, membershipStart) identified edge cases such as today's date, tomorrow's date, that were handled inconsistently across the schema and service layers prior to correction.

These findings are an expected consequence of the BBT approach. By treating the system as a black box and deriving tests purely from the specification, the technique exposes ambiguities that white-box testing, which tends to follow the code paths that exist, cannot easily surface.

White-Box Testing

Overview

This section describes the White-Box Testing (WBT) effort carried out on the backend layer of the GymTracker Pro application, located under gymtrackerpro/lib/.

The testing was performed with full visibility into the internal implementation of each component, deriving test cases directly from the source code structure, control flow, and logical conditions.

Each function under test was analysed by constructing its Control Flow Graph (CFG), computing Cyclomatic Complexity (CC), identifying independent execution paths, and designing test cases to achieve full structural coverage.

Two primary structural testing techniques were applied systematically throughout:

- **Basis Path Testing:** the control flow of each function was modelled as a CFG, from which a basis set of independent paths was derived. Test cases were designed such that each independent path is executed at least once.
- **Multiple Condition Coverage (MCC) with Predicate Coverage:** for every decision node, all possible combinations of condition truth values were exercised, ensuring exhaustive evaluation of logical expressions.

Where loops were present, additional tests were performed, covering execution scenarios from zero iterations up to boundary and beyond-boundary cases.

Scope

Testing covered the entire backend stack across four architectural layers:

Layer	Modules under test
Schema	user-schema, exercise-schema, workout-session-schema, report-schema, utils
Repository	user-repository, exercise-repository, workout-session-repository
Service	user-service, exercise-service, workout-session-service, report-service, auth-service
Controller	user-controller, exercise-controller, workout-session-controller, report-controller, auth-controller
Utilities	escapeLike

Each layer was tested independently, with dependencies at lower layers mocked where applicable (e.g., Prisma client mocked in repository tests; repositories mocked in service tests; services mocked in controller tests), ensuring that each test suite exercises only the internal logic of its own unit.

Test Artifacts

Test Files

A total of 19 test files (.test.ts) were authored, one per module under test.

Test file	Test Count
user-controller.test.ts	44
user-service.test.ts	15
user-repository.test.ts	35
user-schema.test.ts	147
auth-controller.test.ts	7
auth-service.test.ts	5
exercise-controller.test.ts	23
exercise-service.test.ts	7
exercise-repository.test.ts	22
exercise-schema.test.ts	21
workout-session-controller.test.ts	25
workout-session-service.test.ts	6
workout-session-repository.test.ts	44
workout-session-schema.test.ts	65
report-controller.test.ts	4
report-service.test.ts	42
report-schema.test.ts	7
schema/utils.test.ts	22
lib/utils.test.ts	6
TOTAL	**547**

XLSX Documentation Files

Each function under test was accompanied by a structured WBT form in XLSX format. These forms document CFGs (generated using graphviz), cyclomatic complexity calculations, independent paths, and detailed test case coverage.

Layer	Test Count
Schema	20
Repository	24
Service	29
Controller	29
Utilities	1
TOTAL	**103**

Methodology

Control Flow Graph Construction

For each function, a Control Flow Graph (CFG) was constructed to model all possible execution flows.

The CFG consists of:

- A single entry node (START) and a single exit node (EXIT).
- Execution (statement) nodes representing computations.
- Decision nodes representing predicates.
- Merge nodes where control paths reconverge.

Edges represent the flow of execution between nodes, with decision nodes branching into True and False paths.

Cyclomatic Complexity

Cyclomatic Complexity (CC) was computed for each function using three independent methods:

- CC_1 = number of regions in the CFG
- $CC_2 = E - N + 2$ (edges minus nodes plus two)
- $CC_3 = P + 1$ (number of predicate nodes plus one)

All three methods were required to produce the same value, ensuring correctness of the CFG.

The resulting CC value determines:

- The number of independent paths in the basis set
- The minimum number of test cases required for full path coverage

Independent Path Coverage (Basis Set)

From the CFG, a basis set of independent paths was derived.

Each independent path:

- Introduces at least one previously uncovered edge
- Represents a unique execution scenario

Test cases were designed such that:
- Every independent path is executed at least once
- All edges in the CFG are covered

The number of independent paths equals the cyclomatic complexity.

Multiple Condition Coverage (MCC)
For each decision node containing logical conditions, all possible combinations of truth values were tested:
- For a decision with k conditions, 2^k test cases were required
- Each condition was evaluated independently for both True and False outcomes
- Combined with decision coverage, this ensures Decision/Condition Coverage (DCC) and extends to MCC

This approach guarantees that subtle logical faults (e.g., incorrect operator usage, short-circuit errors) are detected.

Predicate Coverage
Predicate coverage was ensured by verifying that:
- Each decision node evaluates to both True and False
- Each path is executed under all relevant predicate outcomes

This complements MCC by ensuring that entire logical expressions influence execution paths correctly.

Loop Testing
For functions containing loops, dedicated test cases were designed to cover iteration behaviour:
- 0 iterations (skip)
- 1 iteration
- 2 iterations
- Typical case (n iterations)
- Boundary cases (n−1, n+1 where applicable)

For nested loops, testing was performed from the innermost loop outward, holding outer loops at minimal values.

Observations and Findings
The structural nature of WBT (547 test cases across 103 forms) revealed several issues that were not apparent from specification-based testing alone:

- Unreachable or redundant code paths. CFG construction exposed branches that could never be executed due to prior conditions, indicating dead code or overly defensive checks.
- Overlapping or subsumed paths. In some cases, multiple independent paths were functionally equivalent, highlighting opportunities for simplification in the implementation.

These findings are a direct consequence of the WBT approach. By analysing the internal structure of the code and systematically exercising all execution paths and condition combinations, WBT exposes defects that specification-based testing cannot detect, particularly those related to control flow, logical correctness, and structural completeness.

Integration Testing

Overview

This document describes the Integration Testing (IT) effort carried out on the backend layer of the GymTracker Pro application, located under gymtrackerpro/lib/.

The testing was performed by progressively combining independently tested modules to verify correct interaction between architectural layers. Unlike Black-Box Testing (BBT), which validates behaviour against specifications, and White-Box Testing (WBT), which validates internal control flow, Integration Testing focuses on module interaction, data flow across layers, and system-level correctness when components are composed together.

The integration strategy follows a **Bottom-Up Incremental Testing** approach, aligned with the layered architecture of the system and its dependency structure as further explained below.

Integration Strategy Rationale

The Bottom-Up Incremental Integration Testing approach was selected as the primary strategy, for the following reasons:

Reduced Test Infrastructure Complexity

In this approach, the Jest testing framework effectively acts as the driver for all integration stages, removing the need for custom driver modules.

Since integration proceeds upward:

- Lower layers are tested with real implementations
- Higher layers are introduced progressively once dependencies are validated
- Mocking is minimized and only used when external isolation is required (e.g., cookies)

This reduces overall test setup complexity compared to top-down or sandwich strategies.

Incremental System Validation

Integration is performed in structured stages:

- **Level 1: Repository Integration** - Repository + Database + Utilities
  Validates database interaction, query correctness, and schema integrity
- **Level 2: Service Integration** - Service + Repository + Database + Utilities
  Validates business logic, computation, and multi-entity coordination
- **Level 3: Controller Integration** - Controller + Validation Schemas + Service + Repository + Database + Utilities
  Validates request handling, validation, error propagation, and full execution flow

This staged approach ensures that each architectural layer is validated in isolation before being combined into full system flows.

Improved Debugging and Fault Isolation

Because modules are integrated incrementally, failures can be traced to a specific layer:

- Repository failures indicate database or query-level issues
- Service failures indicate business logic or data transformation issues
- Controller failures indicate request handling or integration flow issues

This significantly improves debugging efficiency compared to full-system testing approaches.

Trade-off: Execution Time

The Bottom-Up approach introduces a longer overall execution time, as integration tests are executed in multiple sequential stages using a real test database connection.

However, considering all of the above-mentioned benefits, Bottom-Up Integration was chosen over the other Integration Testing techniques.

Scope

Testing covered the backend stack across three integration levels, each progressively composing additional architectural layers:

Layer	Integration Stage	Modules under test
Repository Integration	Repository + Database + Utilities	user-repository, exercise-repository, workout-session-repository, escapeLike
Service Integration	Service + Repository + Database + Utilities	user-service, exercise-service, workout-session-service, report-service, auth-service, user-repository, exercise-repository, workout-session-repository, escapeLike
Controller Integration	Controller + Validation Schemas + Service + Repository + Database + Utilities	user-controller, exercise-controller, workout-session-controller, report-controller, auth-controller, user-schema, exercise-schema, workout-session-schema, report-schema, utils, user-service, exercise-service,

		workout-session-service, report-service, auth-service, user-repository, exercise-repository, workout-session-repository, escapeLike

Test Artifacts

Test Files

A total of 13 test files (.test.ts) were authored, one per module under test.

Test file	Test Count
auth-controller.test.ts	6
auth-service.test.ts	4
exercise-controller.test.ts	33
exercise-service.test.ts	30
exercise-repository.test.ts	29
report-controller.test.ts	6
report-service.test.ts	5
user-controller.test.ts	58
user-service.test.ts	51
user-repository.test.ts	51
workout-session-controller.test.ts	39
workout-session-service.test.ts	35
workout-session-repository.test.ts	35
TOTAL	**382 (~54.27s runtime)**

Methodology

Test Environment and Data

All integration tests execute against a dedicated PostgreSQL test database, separate from development and production environments. The test database is configured via environment variables and managed through Prisma's migration tooling. Each test suite is responsible for seeding the required state before test execution and cleaning up all persisted data after each test or test suite, ensuring full isolation between runs.

The Prisma client used in integration tests is the real production client instance, connected to the test database. No query-level mocking is applied at any integration level, as the purpose of integration testing is to validate real interactions between components and the database.

Interface and Data Flow Verification

Across all integration levels, test cases are designed to verify the correctness of data at each layer interface. Specifically:

- Data passed from a controller to a service must conform to the expected input contract of the service
- Data passed from a service to a repository must conform to the expected query parameters of the repository
- Data returned from the database through the repository to the service and ultimately to the controller must preserve all required fields without unintended transformation or loss

This cross-boundary data flow validation ensures that integration faults, such as missing field mappings, incorrect type coercions, or silently dropped properties, are reliably detected.

Error Propagation Testing

For each integration level, test cases include scenarios in which lower layers produce errors, verifying that:

- Repository errors are correctly handled or re-thrown by service-layer error handling logic
- Service errors are correctly caught by controller-layer error handlers and translated into the expected ActionResult responses and payloads
- No error is silently swallowed at any interface boundary, ensuring full observability of failures across the integrated stack

Observations and Findings

The integration nature of IT (across 382 test cases spanning 3 integration levels) revealed several classes of defects that were not detectable through unit-level WBT or BBT alone:

- **Report computation logic fault**. A defect was identified in the business logic responsible for computing reports, located at the service layer. The fault was not observable in isolation during BBT or WBT, as the unit-level tests operated on mocked repository return values that inadvertently masked the incorrect computation. When the service was integrated with real repository output and a live database, the discrepancy between the expected and actual computed values surfaced, confirming that the fault originated in the service-layer aggregation logic rather than in data retrieval.
- **Confirmed execution time trade-off**. The full integration test suite completed in approximately 54.27 seconds against the real test database. This is consistent with the known trade-off of the Bottom-Up Incremental approach, as each integration level introduces real database I/O, connection management, and sequential transaction overhead that is entirely absent in unit tests. While this represents a meaningful increase

in execution time relative to the vaster BBT and WBT suites, it is an expected and accepted cost given the structural benefits of incremental integration.

These findings are a direct consequence of the integration testing approach. By progressively substituting real implementations for mocked dependencies at each level, integration testing exposes defects that arise specifically from component composition, defects that are structurally invisible to unit testing regardless of the coverage achieved at that level.

Notes

Besides the documented test cases, 16 additional test cases were created in order to cover the Repository and Service singleton creation. These tests focus on internal instantiation guarantees (e.g., instance reuse and dependency consistency) and are therefore considered implementation-level validation rather than functional requirements. As the project documentation primarily describes externally observable system behavior and domain logic, these low-level structural, framework-specific tests were intentionally omitted to maintain clarity and avoid coupling documentation to architectural implementation details.